Computer exercise 2 (mandatory) — Solutions

STA310

> ! Written report
>
> This exercise should be submitted as a written report in Canvas. Use Quarto to produce a PDF. Relevant R-code, output, figures, tables, comments and interpretations should be included in the report. The schedueled session is for questions and discussion, but you will need to work on the exercise in your own time as well!

Relevant packages

```
suppressPackageStartupMessages(library(tidyverse))
library(survival)
```

(You are free to use additional packages as well.)

Kaplan-Meier survival

This is a simple function to estimate Kaplan-Meier survival.

```
#' Kaplan-Meier survival
#' @param time what?
#' @param status what?
#' @return what?
#' @examples
#' time <- c(2, 3, 5, 6, 8)
#' status <- c(1, 1, 0, 1, 0)
#' km(time, status)
#' survfit(Surv(time, status) ~ 1) |> summary()
km <- function(time, status) {
  ts <- sort(unique(time[status == 1]))
  S <- 1
  out <- data.frame(time = ts, n = NA, d = NA, S = NA)
  for (i in seq_along(ts)) {
    t <- ts[i]
    n <- sum(time >= t)
    d <- sum(time == t & status == 1)
    S <- S * (1 - d / n)
```

```
    out$n[i] <- n
    out$d[i] <- d
    out$S[i] <- S
  }
  out
}

# Vectorised version
km2 <- function(time, status) {
  t <- sort(unique(time[status == 1]))
  n <- sapply(t, function(z) sum(time >= z))
  d <- sapply(t, function(z) sum(time == z & status == 1))
  data.frame(time = t, n, d, S = cumprod(1 - d / n))
}
```

1. Run the example (see `@examples` above) to verify similarity between `km()`, `km2()` and the survival package equivalent. Are there any differences?

Solution. The example is run below: `km()` and `km2()` return identical tables, and `summary(survfit(...))` gives the same $\hat{S}(t)$ at the three event times, plus standard errors and confidence intervals. So the point estimates agree; the only difference is that `survfit` additionally reports standard errors and confidence intervals (and marks the censored observations at times 5 and 8).

```
time <- c(2, 3, 5, 6, 8)
status <- c(1, 1, 0, 1, 0)

km(time, status)
```

```
  time n d   S
1    2 5 1 0.8
2    3 4 1 0.6
3    6 2 1 0.3
```

```
km2(time, status)
```

```
  time n d   S
1    2 5 1 0.8
2    3 4 1 0.6
3    6 2 1 0.3
```

```
summary(survfit(Surv(time, status) ~ 1))
```

```
Call: survfit(formula = Surv(time, status) ~ 1)

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    2      5       1      0.8   0.179       0.5161            1
    3      4       1      0.6   0.219       0.2933            1
    6      2       1      0.3   0.239       0.0631            1
```

Include the function definition above (copy and paste either `km()` or `km2()` into a code block in your report) and:

2. Replace "what?" in the function description with relevant information

Solution. The `roxygen`-style description should document the arguments and the return value:

```
#' Kaplan-Meier survival
#' @param time   numeric vector of observed (follow-up) times; censored times
included.
#' @param status event indicator of the same length as `time`: 1 = event, 0 =
right-censored.
#' @return a data.frame with one row per distinct event time and columns `time`
(event
#'   time), `n` (number at risk just before the event), `d` (number of events)
and `S`
#'   (the Kaplan-Meier estimate of the survivor function).
```

3. Add detailed comments within the function to explain what it does (step by step)

Solution. The function walks through the distinct event times in order. At each event time it counts how many subjects are still at risk (`n`) and how many die (`d`), then updates the estimate by the conditional survival fraction $(n - d)/n$. The product of these terms over the event times is the Kaplan-Meier (product-limit) estimator.

```
km <- function(time, status) {
  # Only event times cause a drop; censored times reduce the risk set but no one
  # dies at them, so the estimate is flat there.
  ts <- sort(unique(time[status == 1]))
  S <- 1 # start at S(0) = 1
  out <- data.frame(time = ts, n = NA, d = NA, S = NA)
  for (i in seq_along(ts)) {
    t <- ts[i]
    n <- sum(time >= t) # risk set: subjects still under observation at t
    d <- sum(time == t & status == 1) # events at exactly time t
```

```
    S <- S * (1 - d / n) # product-limit: multiply (n - d)/n
    out$n[i] <- n
    out$d[i] <- d
    out$S[i] <- S
  }
  out
}
```

4. Modify the function to add 95 % confidence intervals to the point estimates. HINT: eq 2.11/2.12 in Collett, `?cumsum` and Wald CI.

Solution. Equation 2.11 is Greenwood's formula for the variance of $\hat{S}(t)$:

$$\operatorname{Var}\left\{\hat{S}(t)\right\} \approx \hat{S}(t)^2 \sum_{t_j \le t} \frac{d_j}{n_j(n_j - d_j)}.$$

The sum is accumulated over event times with `cumsum`, the standard error is its square root, and the Wald 95% interval is $\hat{S}(t) \pm 1.96\,\mathrm{SE}\left\{\hat{S}(t)\right\}$. The function below implements this and is then called on the example data:

```
km_ci <- function(time, status) {
  t <- sort(unique(time[status == 1]))
  n <- sapply(t, function(z) sum(time >= z))
  d <- sapply(t, function(z) sum(time == z & status == 1))
  S <- cumprod(1 - d / n)
  var_S <- S^2 * cumsum(d / (n * (n - d))) # Greenwood (eq 2.12)
  se <- sqrt(var_S)
  data.frame(
    time = t,
    n,
    d,
    S = S,
    SE = se,
    lower = S - 1.96 * se,
    upper = S + 1.96 * se
  )
}

knitr::kable(km_ci(time, status), digits = 3)
```

time	n	d	S	SE	lower	upper
2	5	1	0.8	0.179	0.449	1.151
3	4	1	0.6	0.219	0.171	1.029

time	n	d	S	SE	lower	upper
6	2	1	0.3	0.239	−0.168	0.768

Note that the Wald interval lies outside $[0, 1]$ (the lower bound at time 6 is negative and the upper bound at time 2 exceeds 1). This is why `survfit` uses a log-transformed interval by default (see question 5).

5. Again, compare the results between `km()` (or `km2()`) and the survival package equivalent. Are they the same? If not, why? HINT: check argument `conf.type` in `?survfit.formula` (can you reproduce your results by changing this argument?).

Solution. The point estimates $\hat{S}(t)$ and the standard errors agree with `survfit` - both use Greenwood's formula. The *confidence interval bounds* do **not** match `survfit`'s default output, because `survfit` applies a log transformation (`conf.type = "log"`, the default) before forming the interval, which forces the interval inside $[0, 1]$. To reproduce the Wald interval more closely, use `conf.type = "plain"`:

```
summary(survfit(Surv(time, status) ~ 1, conf.type = "plain"))
```

```
Call: survfit(formula = Surv(time, status) ~ 1, conf.type = "plain")

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    2      5       1      0.8   0.179        0.449        1.000
    3      4       1      0.6   0.219        0.171        1.000
    6      2       1      0.3   0.239        0.000        0.768
```

Bonus: Re-define: `time <- c(2, 2, 1, 3, 2)` and `status <- c(1, 0, 1, 1, 0)` and rerun the analysis. What happened to the SE/CI? Why?

Solution. With the new data the event times are 1, 2 and 3. At the last event time, $t = 3$, the single subject still at risk dies, so $n_3 = d_3 = 1$ and $n_3 - d_3 = 0$. Greenwood's term $d_j/(n_j(n_j - d_j)) = 1/(1 \times 0)$ is then infinite, so the standard error becomes `NaN` and the confidence interval `NA`; $\hat{S}(3) = 0$. `survfit` reports exactly the same thing (`std.err = NaN`, CI `NA` at time 3).

```
time <- c(2, 2, 1, 3, 2)
status <- c(1, 0, 1, 1, 0)

knitr::kable(km_ci(time, status), digits = 3)
```

time	n	d	S	SE	lower	upper
1	5	1	0.8	0.179	0.449	1.151
2	4	1	0.6	0.219	0.171	1.029

time	n	d	S	SE	lower	upper
3	1	1	0.0	NaN	NaN	NaN

```
summary(survfit(Surv(time, status) ~ 1, conf.type = "plain"))
```

```
Call: survfit(formula = Surv(time, status) ~ 1, conf.type = "plain")

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    1      5       1      0.8   0.179        0.449            1
    2      4       1      0.6   0.219        0.171            1
    3      1       1      0.0     NaN          NaN          NaN
```

The reason is that Greenwood's variance treats each event time like a binomial proportion d/n; when *everyone* left in the risk set dies ($d = n$), there is no remaining information with which to estimate the variance - the curve is forced down to 0 and its uncertainty is undefined. This is a reminder that the plain Wald interval on S is unstable near the extremes, which is why transformed intervals (log or log-log) are used in practice.

Kaplan-Meier plot

> **Tip**
>
> Reset `time` and `status` to the original example data if you have changed them in the previous bonus question.

Create a simple plot function:

```
plot_km <- function(x) ggplot(x, aes(time, S)) + geom_step()
```

Compare `plot_km(km(time, status))` and `plot(survfit(Surv(time, status) ~ 1), conf.int = FALSE)`.

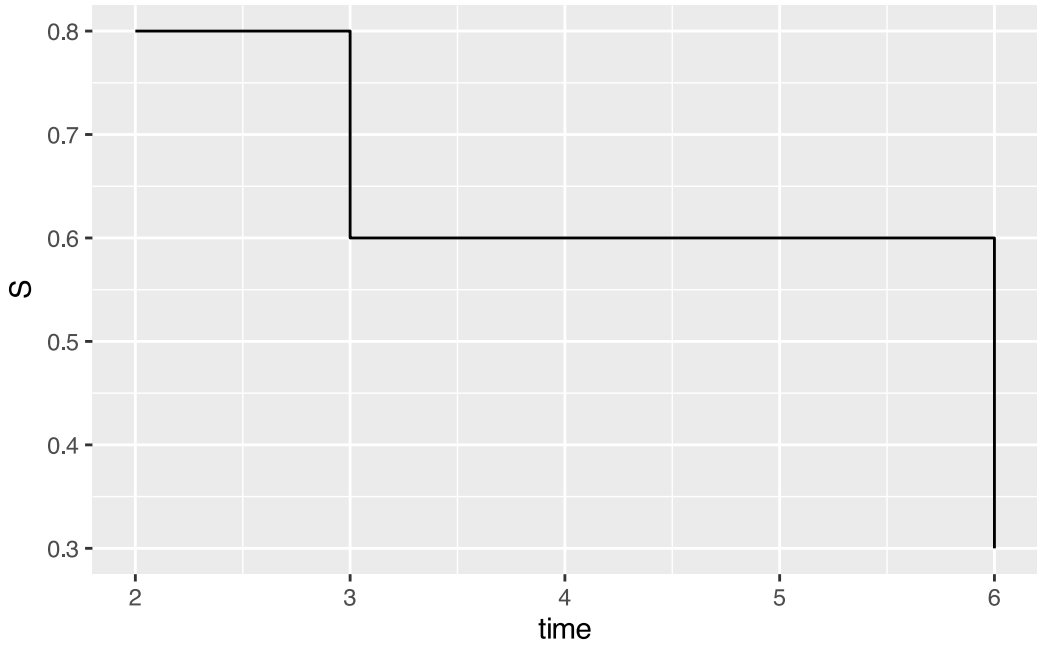
6. How and why do the results differ (if they do)?

Solution.

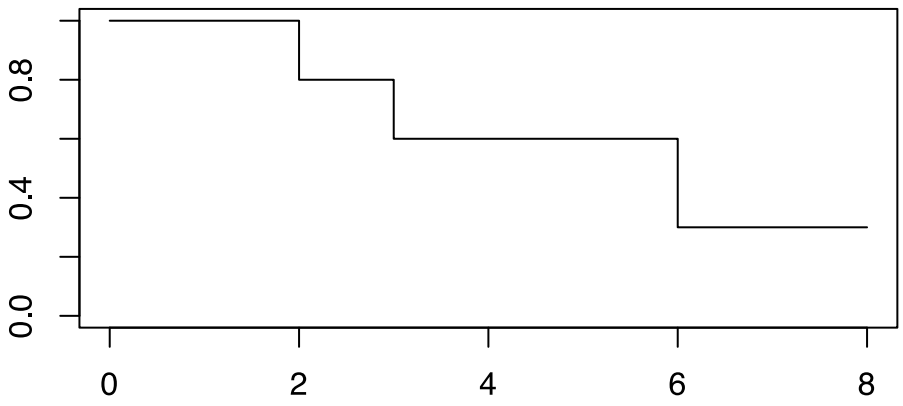
- `plot_km(km(time, status))` starts at the first event time ($t = 2, S = 0.8$); it has no flat segment at $S = 1$ from time 0 to the first event.
- It does not extend to the last observation time (8).

```
time <- c(2, 3, 5, 6, 8) # re-set to the original example data
status <- c(1, 1, 0, 1, 0)
```

```r
plot_km(km(time, status))
```




```r
plot(survfit(Surv(time, status) ~ 1), conf.int = FALSE)
```



7. Explain (in words) how you would improve your functions (`km()`, `plot_km()` or both) to make your plot look like a real Kaplan-Meier plot.

Solution. To make the plot look like a proper Kaplan-Meier plot you would:

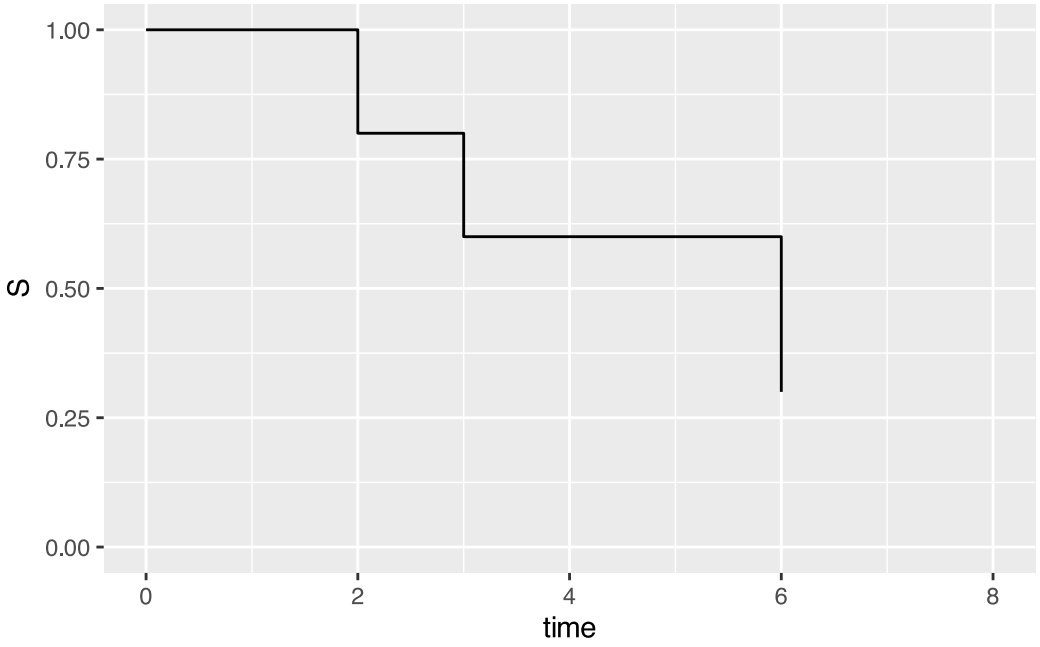
- add a point $(0, 1)$ so the curve starts at 1 and is flat until the first event;
- extend the step to the largest observed (possibly censored) time;

Bonus: Implement your suggested changes from (7) and check the result.

Solution. One implementation (base `ggplot2` only):

```
km3 <- function(time, status) {
  t <- sort(unique(time[status == 1]))
  n <- sapply(t, function(z) sum(time >= z))
  d <- sapply(t, function(z) sum(time == z & status == 1))
  data.frame(
    time = c(0, t),
    n = c(length(time), n),
    d = c(0, d),
    S = c(1, cumprod(1 - d / n))
  )
}

plot_km(km3(time, status)) +
  coord_cartesian(ylim = c(0, 1), xlim = c(0, max(time)))
```



(In practice the cleanest route is `ggsurvfit::ggsurvfit(survfit(...))`, which produces a fully annotated plot automatically.)

Cox

Here is a simple Cox function for one covariate `x`:

```
cox <- function(time, status, x, beta = 0, tol = 1) {
  repeat {
    U <- I <- 0 # score and information
    for (i in which(status == 1)) {
      R <- which(time >= time[i]) # Risk set
      w <- exp(beta * x[R])
      mean_x <- sum(w * x[R]) / sum(w)
      mean_x2 <- sum(w * x[R]^2) / sum(w)
      U <- U + x[i] - mean_x # Score
      I <- I + mean_x2 - mean_x^2 # Information
    }
    beta_new <- beta + U / I # Newton-Raphson
    if (abs(beta_new - beta) < tol) {
      break
    }
    beta <- beta_new
  }

  c(beta = beta_new, SE = sqrt(1 / I))
}
```

> Tip
>
> Recapture section 3.3 from the book.

8. As for `km()` (`km2()`), include the function definition above (copy and paste into a code block in your report) and add a description of the function and comments within the function to explain what it does in more details.

Solution. The function maximises the *partial likelihood* of a Cox proportional hazards model with a single covariate `x`, using the Newton-Raphson algorithm. `U` is the score (first derivative of the partial log-likelihood), `I` the observed information (second derivative), and the estimate is updated by $\beta_{\text{new}} = \beta + U/I$; at convergence the standard error is $\text{SE} = 1/\sqrt{I}$.

```
cox <- function(time, status, x, beta = 0, tol = 1) {
  # Newton-Raphson iterations:
  # repeatedly update beta until the change in beta is sufficiently small
```

```
repeat {
  # U(beta) = score = first derivative of the partial log-likelihood
  # I(beta) = observed information = - second derivative of the
  # partial log-likelihood
  #
  # Both are recomputed at the current value of beta in each iteration.
  U <- I <- 0

  # Loop over individuals who experience the event.
  #
  # If several individuals have an event at the same time, each contributes
  # separately but uses the SAME risk set. This corresponds to Breslow's
  # handling of tied event times.
  for (i in which(status == 1)) {
    # Risk set R_i:
    # all individuals still under observation and at risk immediately
    # before the event time of individual i.
    R <- which(time >= time[i])

    # Relative hazard for each individual in the risk set:
    #
    #     exp(beta * x_j)
    #
    # These are the weights appearing in the denominator of the
    # Cox partial likelihood.
    w <- exp(beta * x[R])

    # Risk-set weighted mean of x:
    #
    #       sum x_j exp(beta*x_j)
    # E(x) = ---------------------
    #         sum exp(beta*x_j)
    #
    # This is the expected value of x among individuals in the risk set,
    # weighted according to their current relative hazards.
    mean_x <- sum(w * x[R]) / sum(w)

    # Corresponding risk-set weighted mean of x^2.
    mean_x2 <- sum(w * x[R]^2) / sum(w)

    # Contribution to the score:
    #
    #     observed x for the individual with the event
```

```
    #                   -
    #     weighted expected x in the risk set
    #
    # Summing these contributions gives U(beta).
    # The MLE is the beta for which U(beta) = 0.
    U <- U + x[i] - mean_x

    # Contribution to the observed information:
    #
    #     E(x^2) - E(x)^2
    #
    # i.e. the weighted variance of x within the risk set.
    #
    # This is -d^2 l(beta)/d beta^2 for this event contribution.
    I <- I + mean_x2 - mean_x^2
  }

  # Newton-Raphson update.
  #
  # In general:
  #
  #   beta_new = beta - score / second derivative
  #
  # Since I = - second derivative here, this becomes
  #
  #   beta_new = beta + U / I
  #
  # Thus we move beta toward the value where the score U(beta) = 0.
  beta_new <- beta + U / I

  # Convergence criterion:
  # stop when the Newton-Raphson update changes beta by less than tol.
  if (abs(beta_new - beta) < tol) {
    break
  }

  # Use the updated estimate as the starting value for the next iteration.
  beta <- beta_new
}

# At convergence:
#
# beta_new is the maximum partial-likelihood estimate of beta.
```

```
  #
  # The approximate variance follows from the curvature of the
  # partial log-likelihood:
  #
  #     Var(beta_hat) ≈ 1 / I(beta_hat)
  #
  # and hence
  #
  #     SE(beta_hat) ≈ sqrt(1 / I(beta_hat)).
  c(
    beta = beta_new,
    SE = sqrt(1 / I)
  )
}
```

Define a covariate `x <- c(0, 1, 0, 1, 1)`.

9. Compare `cox(time, status, x)` to the equivalent analysis performed by functions from the survival package.

Solution. With the default `tol = 1`, `cox(time, status, x)` returns $\hat{\beta} \approx -0.819$ and $\mathrm{SE} \approx 1.529$, whereas `coxph` returns $\hat{\beta} = -0.752$ with $\mathrm{SE} = 1.436$ - the two do not agree.

```
time <- c(2, 3, 5, 6, 8) # re-set to the original example data
status <- c(1, 1, 0, 1, 0)
x <- c(0, 1, 0, 1, 1)

cox(time, status, x)
```

```
      beta         SE
-0.8187135  1.5294382
```

```
coxph(Surv(time, status) ~ x)
```

```
Call:
coxph(formula = Surv(time, status) ~ x)

     coef exp(coef) se(coef)      z   p
x -0.7520    0.4714   1.4355 -0.524 0.6

Likelihood ratio test=0.27  on 1 df, p=0.6042
n= 5, number of events= 3
```

10. Do the results differ? If so, why? HINT: what is the purpose of argument `tol` (you may find `?coxph.control` relevant)? Try different values of `tol` and check if the results from `cox()` and `coxph()` can reach agreement.

Solution. They differ because of the loose convergence tolerance. With `tol = 1` the Newton-Raphson loop stops as soon as $|\beta_{\text{new}} - \beta| < 1$. Starting at $\beta = 0$, the first update is $\beta_{\text{new}} = U/I \approx -0.819$, and since $|-0.819 - 0| < 1$ the loop breaks after a single step, returning an estimate that has not converged to the maximum of the partial likelihood (and a standard error evaluated at the wrong point). `coxph` instead iterates to convergence using a much tighter criterion (see `?coxph.control`; the default `eps = 1e-9`) based on the log-likelihood.

The purpose of `tol` is precisely the convergence tolerance of the Newton-Raphson update. Setting it small makes the two agree:

```
cox(time, status, x, tol = 1e-8)
```

```
      beta         SE
-0.7520387  1.4355000
```

Analysis

We will use the `lung` data from the survival package. Read about it: `?lung`.

Try `with(lung, cox(time, status - 1, ph.karno))`.

11. Formulate a research question associated with the analysis (the code you tried).

Solution. For example: *"Is a patient's Karnofsky performance score at diagnosis (`ph.karno`) associated with survival in the North Central Cancer Treatment Group (NCCTG) lung cancer cohort, and how much does the hazard of death change per unit increase of the score?"* Equivalently: does better baseline performance status predict longer survival?

12. Did you get any result? If not, why? HINT: perform the same analysis with the equivalent survival package function and read the output carefully

Solution. No - `cox()` errors with `missing value where TRUE/FALSE needed`, because `ph.karno` contains a missing value (the `lung` data has 228 rows and one `NA` in `ph.karno`). The `NA` propagates through the weighted means into the score and information, so `beta_new` becomes `NA` and `abs(NA - beta) < tol` cannot be evaluated. `coxph` handles this by dropping incomplete cases: it reports `n = 227, number of events = 164 (1 observation deleted due to missingness)` - that line is the clue.

```
try(with(lung, cox(time, status - 1, ph.karno)))
```

```
Error in if (abs(beta_new - beta) < tol) { :
  missing value where TRUE/FALSE needed
```

```
coxph(Surv(time, status - 1) ~ ph.karno, data = lung)
```

```
Call:
coxph(formula = Surv(time, status - 1) ~ ph.karno, data = lung)

              coef exp(coef)  se(coef)     z       p
ph.karno -0.016448  0.983686  0.005854 -2.81 0.00496

Likelihood ratio test=7.56  on 1 df, p=0.005966
n= 227, number of events= 164
   (1 observation deleted due to missingness)
```

13. Fix the problem and check that the result from `cox()` is equivalent to the result from the equivalent survival package function.

Solution. Remove the incomplete case before calling `cox()`, and (per question 10) use a tight tolerance:

```
cc <- lung[complete.cases(lung$ph.karno), ]

cox(cc$time, cc$status - 1, cc$ph.karno, tol = 1e-8)
```

```
        beta           SE
-0.016424495  0.005854927
```

```
coxph(Surv(time, status - 1) ~ ph.karno, data = cc)
```

```
Call:
coxph(formula = Surv(time, status - 1) ~ ph.karno, data = cc)

              coef exp(coef)  se(coef)     z       p
ph.karno -0.016448  0.983686  0.005854 -2.81 0.00496

Likelihood ratio test=7.56  on 1 df, p=0.005966
n= 227, number of events= 164
```

The two now agree (to within the numerical tolerance).

14. Interpret the result and summarise your findings (using statistical, as well as clinical language).

Solution. The estimated log hazard ratio is -0.0164 (SE 0.0059), so the hazard ratio per one-point increase in Karnofsky score is $\exp(-0.0164) \approx 0.984$ with 95% CI $(0.972, 0.995)$; $z = -2.81$, $p = 0.005$.

```r
cc <- lung[complete.cases(lung$ph.karno), ]
m0 <- coxph(Surv(time, status - 1) ~ ph.karno, data = cc)

exp(coef(m0)) # hazard ratio
```

```
 ph.karno
0.9836863
```

```r
exp(confint(m0)) # 95% CI for the hazard ratio
```

```
             2.5 %    97.5 %
ph.karno 0.9724645 0.9950377
```

```r
exp(10 * coef(m0)) # hazard ratio for a 10-point difference
```

```
 ph.karno
0.8483329
```

Statistically: the confidence interval excludes 1, so there is strong evidence of an association - a higher Karnofsky score is associated with a lower hazard of death.

Clinically: each additional point of performance status is associated with about a 1.6% lower hazard. Over a 10-point difference this compounds to about a 15% lower hazard for a patient scored 10 points higher. Patients in better functional condition at diagnosis survive longer.

15. How would you improve the analysis? You may reformulate the research question and you may use any available tool (methods/packages) for the analysis.

Solution. Several improvements are possible:

- **Adjust for confounders.** The univariate model ignores other prognostic variables; fit a multivariable model (e.g. add `age` and `sex`).
- **Do not double-count performance status.** `ph.karno` and `ph.ecog` measure the same underlying construct (their correlation is about -0.81); include only one of them.
- **Check the functional form.** The model assumes the log hazard is *linear* in `ph.karno`; consider treating it as categorical or checking linearity.
- **Handle missing data deliberately.** Complete-case analysis drops one row; multiple imputation would use all observations.
- **Check the proportional-hazards assumption.** `cox.zph()` suggests `ph.karno` violates proportional hazards, i.e. its effect may weaken over time, so the single hazard ratio is a time-average.

16. Perform your suggested analysis and present your results with relevant figures, tables and/or summaries/interpretations.

Solution. A multivariable model adjusting for age and sex:

```
fit <- coxph(Surv(time, status - 1) ~ ph.karno + age + sex, data = lung)
gtsummary::tbl_regression(fit, exponentiate = TRUE)
```

Characteristic	HR	95% CI	p-value
ph.karno	0.99	0.98, 1.00	0.023
age	1.01	0.99, 1.03	0.2
sex	0.61	0.44, 0.84	0.003

Abbreviations: CI = Confidence Interval, HR = Hazard Ratio

```
modelsummary::modelsummary(fit, exponentiate = TRUE)
```

	(1)
ph.karno	0.987
	(0.006)
age	1.012
	(0.010)
sex	0.608
	(0.102)
Num.Obs.	227
AIC	1476.2
BIC	1486.4
RMSE	0.90

```
cox.zph(fit)
```

```
          chisq df      p
ph.karno  8.017  1 0.0046
age       0.478  1 0.4892
sex       3.085  1 0.0790
GLOBAL   10.359  3 0.0157
```

After adjusting for age and sex, the association of `ph.karno` persists but decrease in size. Being male roughly doubles the hazard relative to female (HR 0.608), while age is not clearly associated.

A caveat: `cox.zph(fit)` indicates `ph.karno` may violate proportional hazards, so these hazard ratios should be read as time-averaged effects. A fuller analysis would also plot a Kaplan-Meier curve stratified by performance-status groups and report the proportional-hazards diagnostics.